

Fly GACA — Source-update runbook

SECTION 06-operations-it

TYPE runbook

STATUS **active**

OWNER Founder

UPDATED 2026-07-03

How Fly GACA keeps the **GACAR**, the **AIP** and the rest of the GACA corpus in sync with GACA's own publications — automatically, on the AIRAC cadence.

TL;DR

A scheduled GitHub Action checks GACA's official pages, downloads anything new, refreshes the indexes, guards data integrity, and commits the result. You don't have to do anything for routine cycles.

```
# Manually, from a machine that can reach gaca.gov.sa:
npm run check:sources      # is anything new on GACA? (no writes; exit 1 if drift)
npm run update:sources     # download new docs + refresh indexes + status file
```

What's wired

Piece	Path	Role
Source manifest	<code>assets/data/sources.json</code>	The official GACA URLs, the AIRAC config, and the per-source fingerprints. The updater owns the fingerprints — don't hand-edit them.
Updater engine	<code>scripts/update-sources.js</code>	Fetches each GACA page, discovers the published documents, detects change, downloads new files, refreshes indexes. Zero dependencies.
Freshness file	<code>assets/data/source-status.json</code>	Auto-generated public snapshot — last checked, current/next AIRAC, per-source status. Safe to ship; the app can show "GACAR current as of ...".
Schedule	<code>.github/workflows/update-sources.yml</code>	Weekly (Mon) + every Thursday (to catch AIRAC effective dates) + manual dispatch.

Sources tracked (the "everything from GACA" set)

id	What	URL
<code>gacar</code>	GACAR Parts & framework	https://gaca.gov.sa/en/Rules-and-Regulations-Category
<code>aip</code> / <code>aerodromes</code> / <code>charts</code>	AIP / eAIP, AD 2, VFR charts (AIRAC-bound)	https://gaca.gov.sa/web/en/aeronautical-information

id	What	URL
<code>advisory-circulars</code>	AC-xxx	https://gaca.gov.sa/web/en/advisory-circulars
<code>notam</code>	NOTAM / PIB (time-sensitive)	https://gaca.gov.sa/web/en/notam

The AIRAC cadence

AIP currency follows the 28-day AIRAC cycle. The updater computes the current and next effective dates from the anchor in `sources.json` (`airac.anchor` , `airac.cycleDays`). AIRAC-bound sources stamp their index `generated` date with the **effective AIRAC date**, not the run date, so the published date matches the cycle. The workflow's Thursday cron lines up with AIRAC Thursdays.

How a run flows

1. Fetch each enabled GACA page (with redirects, retries, a 25s timeout).
2. Discover every document link (PDF / office formats) and fingerprint the set.
3. Compare to the recorded fingerprint → changed?
4. On change (`--apply`): download the new files into `library/<staging>/` , run `pdftotext` on PDFs when available, bump the affected index's `generated` date, update the fingerprint, and rewrite `source-status.json` .
5. `node scripts/check-data.js` gates the result — a corrupted or count-drifted data set fails the run before anything is committed.
6. Commit `assets/data/` and push. (Optionally deploy — see below.)

Fail-safe behaviour

- **A blocked or empty fetch never wipes content.** If a page that previously had documents suddenly lists zero, the updater treats it as a fetch error and changes nothing.
- `--check` **only fails on a positive change.** Network trouble is a soft warning (exit 0), so the schedule never pages anyone over a flaky fetch.
- `library/` **is git-ignored staging.** Only the processed `assets/data/` is committed and deployed (per `library/README.md`). Downloaded PDFs live in the runner for that run's extraction; they are not committed.

Making publication fully automatic (optional deploy)

Deploys are otherwise manual (`firebase deploy`), and a push made by the workflow's default token does **not** re-trigger `ci.yml` 's deploy job. So to let this workflow publish on its own, add a repository secret `FIREBASE_TOKEN` (from `firebase login:ci`). The workflow's final step deploys Hosting only when that secret is present and a change was committed. Without the secret, the data is committed and the next normal deploy ships it.

First live run — tuning the scraper

This pipeline could not be exercised against `gaca.gov.sa` from the build sandbox (its network allowlist returns `403 Host not in allowlist` for that host), so link discovery is a tolerant best-effort: it pulls every `<a href>` ending in `.pdf/.doc/.docx/.xls/.xlsx/.zip`. On the first real run, check the workflow log's per-source `N docs` counts:

- **0 docs on a source that should have many** → GACA serves that list via a format the regex misses (e.g. links without a file extension, or an API/JSON feed). Widen the `EXT` pattern or the discovery logic in `scripts/update-sources.js` (`discover()`), or point the source `url` at the page that actually lists the files.
- The updater's fail-safe means a mis-tuned selector reports "0 / error" and changes nothing — it won't damage the live data while you adjust it.

Deep content extraction (the plug-in point)

The updater gets the **source files** and keeps the **indexes' freshness** in sync automatically. Turning a refreshed PDF into the per-Part reading HTML (`assets/data/parts/*.html`) and the whole-library search index is the one step that needs GACA's actual document structure (see `office/content-integration-plan.md`). Add that transform as `scripts/extract-sources.js` and call it from the workflow between the fetch and the integrity guard; the counts in `check-data.js` and the README must move with it (they're contractual).

Operational runbook — not legal advice. GACA remains the authoritative source.